Procyberian Systems Distribution (PSD) - Dosya Sistemi ve İşletim Sistemi Geliştirme Kılavuzu

Bu doküman, özgün GNU/Linux dağıtımınız **PSD (Procyberian Systems Distribution)** için C, C++ ve GNU Assembler (gas) kullanarak geliştirmeyi planladığınız özel dosya sistemine dair şu ana kadar konuşulan tüm teknik temelleri, mimari gereksinimleri ve yol haritasını tek bir çatı altında toplamaktadır.

---

1. İşletim Sisteminde Dosya Sistemi Nedir?

Dosya sistemi, bir işletim sisteminin verileri sabit disk veya SSD gibi depolama birimleri üzerinde nasıl saklayacağını, organize edeceğini ve bulacağını belirleyen **yöntem ve kurallar bütünüdür.**

Dosya Sisteminin Temel Görevleri

- **Veri Düzenleme:** Dosyaları klasörler (dizinler) halinde hiyerarşik bir yapıda tutar.
- **Adlandırma:** Dosyalara isim verir ve bu isimlerin kurallarını belirler.
- **Meta Veri Yönetimi:** Dosyaların boyutu, oluşturulma tarihi, değiştirilme zamanı ve erişim izinleri (permissions) gibi öznitelikleri saklar.
- **Yer Belirleme:** Verilerin disk üzerinde hangi fiziksel bloklara yazılacağını ve buralardan nasıl okunacağını kontrol eder.

Yaygın Dosya Sistemi Örnekleri

- **NTFS:** Windows işletim sistemlerinin modern ve standart dosya sistemidir.
- **FAT32 / exFAT:** Taşınabilir belleklerde (USB, SD kart) sıkça kullanılan, farklı işletim sistemleriyle yüksek uyumluluğa sahip sistemlerdir.
- **EXT4:** Linux tabanlı sistemlerde yaygın olarak kullanılan, günlükleme (journaling) desteğine sahip kararlı bir dosya sistemidir.
- **APFS:** Apple'ın macOS ve iOS cihazları için geliştirdiği güncel dosya sistemidir.

---

2. Bir Dosya Sistemi Geliştirmek İçin Gereken Bilgiler

Bir dosya sistemi geliştirmek; hem matematiksel teorilere hem de doğrudan donanımla konuşan pratik mühendislik bilgilerine hakim olmayı gerektirir.

A. Matematiksel Temeller

Dosya sistemleri, verileri en az yer kaplayacak ve en hızlı bulunacak şekilde yerleştirmek için matematiksel modelleri kullanır: * **Graf Teorisi (Graph**

Theory): Klasörlerin iç içe geçmesi ve birbirine bağlanması bir ağaç (tree) yapısı oluşturur. Dizin hiyerarşisini yönetmek için graf teorisi algoritmaları şarttır. * **Kombinatorik ve Olasılık:** Diskteki boş alanların verimli dağıtılması, fragmantasyonun (parçalanma) önlenmesi ve veri kaybı olasılıklarının hesaplanmasında kullanılır. * **Kriptografi (Şifreleme Bilimi):** Modern dosya sistemlerinde verilerin güvenliği ve disk düzeyinde şifrelenmesi (encryption) için karmaşık matematiksel algoritmalar gerekir.

B. Veri Yapıları ve Algoritmalar

Dosya sisteminin kalbi, veriyi disk üzerinde nasıl modellediğiniz ve aradığınızdır: * **B-Ağaçları (B-Trees / B+ Trees):** Milyonlarca dosya arasından aradığınız bir dosyayı nanosaniyeler içinde bulmanızı sağlayan, disk tabanlı depolamaya en uygun matematiksel veri yapısıdır. * **Bağlı Listeler (Linked Lists):** Bir dosyanın parçalarının (bloklarının) diskin hangi köşelerine dağıldığını (cluster/block chaining) takip etmek için kullanılır. * **Karma Tabloları (Hash Tables):** Dosya adlarını veya dizin yollarını hızlıca diskin fiziksel adreslerine veya inode numaralarına dönüştürmek için kullanılır.

C. Bilgisayar Mimarisi ve İşletim Sistemleri

Matematiksel modelleri gerçeğe dönüştürmek için donanımı alt seviyede tanımanız gerekir: * **Alt Seviye Programlama:** C veya Rust gibi donanıma doğrudan erişebilen dilleri ve işlemci mimarisine özgü Assembly yönergelerini bilmelisiniz. * **Disk Geometrisi:** Sabit disklerin (HDD) sektör yapısı ile modern katı hal sürücülerinin (SSD) çalışma mantığı (aşınma dengeleme - wear leveling, blok silme) tamamen farklıdır. Sürücünün ömrünü ve performansını optimize edecek algoritmalar kurulmalıdır. * **Önbellek Yönetimi (Caching):** Diske yazma ve okuma işlemleri RAM'e göre çok yavaştır. RAM'i bir köprü (buffer cache) gibi kullanarak bu işlemleri hızlandırma algoritmaları (LRU vb.) uygulanmalıdır.

D. Hata Toleransı ve Güvenilirlik

- **Günlükleme (Journaling):** Bilgisayarın fişi aniden çekildiğinde veya sistem çöktüğünde verilerin bozulmaması (corruption) için, yapılan işlemlerin diske kalıcı olarak yazılmadan önce bir günlüğe (journal) kaydedilmesi mantığı bilinmelidir.

---

3. C, C++ ve GNU Assembler (GAS) ile Alt Seviye Geliştirme

Hız ve donanım kontrolü gerektiren kritik yerlerde Assembly (GAS), ana mimari ve nesne yönelimli tasarımda ise C/C++ dillerinin birlikte kullanımı en güçlü yaklaşımdır.

Kritik Entegrasyon Konuları

1. **Satır İçi Assembly (Inline Assembly):** C/C++ kodunun içerisine `__asm__` anahtar kelimesiyle küçük Assembly blokları ekleyerek, işlemci kayıtçılarına (register) doğrudan ulaşabilir ve disk kontrolcülerine (I/O ports) doğrudan komut gönderebilirsiniz.
2. **Ayrı Dosyaları Birleştirme (Linking):** Donanıma yakın fonksiyonları `.s` uzantılı GNU Assembler dosyalarında, üst seviye mantığı `.cpp` dosyalarında yazıp, **GCC** ve **Linker (ld)** yardımıyla tek bir ikili (binary) dosyada birleştirebilirsiniz.
3. **Çağrı Standartları (Calling Conventions):** C++ fonksiyonlarından Assembly fonksiyonlarına veri (parametre) aktarırken, ilgili mimarinin (örneğin x86_64) kurallarına (System V AMD64 ABI: parametrelerin sırasıyla RDI, RSI, RDX kayıtçılarına aktarılması gibi) harfiyen uyulmalıdır.
4. **`extern "C"` Blokları:** C++ derleyicisinin fonksiyon isimlerini arka planda değiştirmesini (*Name Mangling*) engellemek ve Assembly tarafının C++ fonksiyonlarını net bir şekilde görebilmesini sağlamak için bu deklarasyon kullanılmalıdır.

---

4. PSD (Procyberian Systems Distribution) İçin Yol Haritası

Özgün GNU/Linux dağıtımınızı bu özel dosya sistemiyle sunmak ve sistemin bu diskten önyükleme yapabilmesini sağlamak için izlenecek mimari adımlar:

A. Linux Çekirdeği (Kernel) Entegrasyonu

- **VFS (Virtual File System) Arayüzü:** Linux, tüm dosya sistemlerini soyutlayan bir sanal katman kullanır. Yazacağınız C kodlarının, Linux çekirdeğindeki `inode_operations`, `file_operations` ve `super_operations` gibi VFS yapılarını (struct) doldurması gerekir.
- **Kernel Modülü (LKM):** İlk aşamada kodları doğrudan çekirdeğe gömmek yerine, dinamik olarak yüklenebilen bir `.ko` (Kernel Object) modülü olarak geliştirmek test ve hata ayıklama (debugging) süreçlerini kolaylaştırır.

B. Başlangıç Süreci (Boot) ve Kurulum

- **Bootloader Desteği:** GRUB gibi bir ön yükleyicinin açılışta diski okuyabilmesi gerekir. GRUB'a kendi dosya sisteminizi okuyabilmesi için C ile yazılmış mini bir sürücü ekleyebilir veya `/boot` bölümünü standart ext4 yapıp ana kök (`/`) sisteminizi kendi dosya sisteminizde tutabilirsiniz.
- **initramfs Kurulumu:** Çekirdek ilk yüklendiğinde diskinizi doğrudan bağlayamaz. Bellekte açılan geçici bir mini dosya sistemi (initramfs) içine

kendi dosya sistemi sürücü modülünüzü koymalısınız ki, sistem açılırken diskinizi bağlayıp (`mount`) ana süreci başlatabilsin.

C. Kullanıcı Alanı Araçları (User-space Tools)

Kullanıcıların PSD üzerinde diski yönetmesi için C/C++ ile şu araçların yazılması gerekir: * **`mkfs.psd`:** Depolama birimini Procyberian dosya sistemine göre biçimlendiren araç. * **`fsck.psd`:** Hata durumlarında dosya bütünlüğünü tarayıp tamir eden araç.

D. Geliştirme Önerisi: FUSE (Filesystem in Userspace)

Doğrudan Linux çekirdeği (Kernel space) içinde kod yazarken yapılacak küçük bir hata sistemi çökerteceğinden (*Kernel Panic*), projenin ilk aşamalarında dosya sisteminin mantığını ve C++ kodlarını **FUSE** teknolojisiyle geliştirmek en güvenli yoldur. FUSE sayesinde dosya sisteminizi normal bir kullanıcı programı gibi yazıp test edebilir, kararlı hale geldiğinde çekirdek seviyesine taşıyabilirsiniz.